Projeto Agrupamento Utilizando Formigas

Gustavo de Souza[1], **Jose Augusto Laube**[2]

[1]Universidade do Estado de Santa Catarina (UDESC)
Departamento de Ciência da Computação
Joinville – SC – Brasil CEP: 89.219-710

gustavo.souza11@edu.udesc.br, jose.laube17@edu.udesc.br

Resumo. *Este trabalho apresenta a implementação de um sistema de agrupamento inspirado no comportamento necrófilo de formigas reais. Na primeira fase aqui relatada, focou-se no agrupamento de itens homogêneos em um ambiente toroidal. Utilizou-se agentes reativos com regras de probabilidade baseadas na densidade local para organizar itens distribuídos aleatoriamente em aglomerados coesos. Os resultados demonstram a emergência de ordem a partir de regras simples de interação local.*

1. Introdução

O algoritmo de agrupamento por formigas é inspirado no comportamento real dentro de um formigueiro, em que as elas criam cemitérios para separar as formigas mortas do resto da colônia, assim evitando a proliferação de pragas e doenças. Algoritmos inspirados no comportamento de formigas têm sido amplamente estudados na literatura, especialmente no contexto de auto-organização e formação de padrões emergentes [Deneubourg et al. 1991, Lumer and Faieta 1994], e desde então vem sendo usado como uma alternativa ao método k-means. Portanto o algoritmo de colônia de formigas para agrupamento é uma solução de auto-organização. Ele prova que regras locais simples podem gerar uma ordem global complexa e eficiente.

2. Metodologia de Desenvolvimento

O trabalho foi desenvolvido utilizando a linguagem de programação C, foram feitos um struct para os agentes (formigas vivas), um struct para os dados (formigas mortas), e funções que permitisse dar a vida para os agentes. A liguaguem C foi escolhida pelo maior dominio da equipe, e pois inicialmente desejavamos fazer em paralelo, contudo optamos por manter o algoritmo de forma sequêncial.

2.1. Ambiente e Agentes

Para o ambiente foi definido inicialmente uma matriz de 50 por 50, inicialmente com barreiras nas bordas, para observarmos se o comportamento que definimos para os agente estava correto, depois foi feito a união das bordas, assim o espaço fica similar ao de um toroide, pensando de forma tridimensional, onde não tem bordas, assim se o agente for para um lado que saí do espaço da matriz, ele volta do outro lado do ambiente.

Os agentes foram definidos por um struct, que salva a posição em x (posiX), a posição em y (posiY), o status se a formiga está carregando ou não um dado e o id que representa esse dado, assim como se a formiga morta estiver sendo carregada ela não é detectada pelos outros agentes.

2.2. Movimentação e Visão

O movimento dos agentes é estocastico, podendo ser para esquerda, direita, cima ou baixo, uma função que gera números aleatórios de forma uniforme, sorteia para que direção o agente vai. Já a função visão vai olhar a quantidade de dados tem em sua volta, sempre considerando o raio que foi definido, para os dados homogêneos a função apenas conta quantos dados tem a sua volta e calcula a probabilidade da formiga pegar/largar o item de uma maneira simples, conta a quantidade de itens nas casas a sua volta de acordo com o raio de visão, já para os dados heterogêneos, a função calcula a distância euclidiana entre os dados a sua volta, para dar a probabilidade do dado ser pego/solto ou não, as funções probabilísticas de pegar e soltar itens seguem o modelo clássico de comportamento de formigas descrito por Deneubourg et al. [Deneubourg et al. 1991].

2.3. Regras de Transição

A transição das formigas ocorre sempre analisando a sua posição atual e seu status, ou seja, se a formiga estiver carregando e em uma posição que já tem um dado, ela anda, o mesmo se ela não está carregando e em uma posição que não tem um dado. Agora caso ela possa pegar ou largar um item ela vai usar as probabilidades que foram dadas pela função visão, o algoritmo vai gerar uma probabilidade aleatória, distribuíção uniforme, e se essa probabilidade for menor ou igual a probabilidade retornada, o agente vai executar a sua ação de pegar/largar.

3. Descrição de Experimentos e Resultados

Para validar a evolução conceitual e garantir a integridade da lógica de Lumer-Faieta, a implementação do projeto foi deliberadamente dividida em duas etapas. Todo o código-fonte desenvolvido, juntamente com as diferentes versões, histórico de commits e os *datasets* utilizados (`dados_4.txt` e `dados_15.txt`), encontra-se versionado e disponível no repositório oficial do projeto no GitHub: `[https://github.com/JoseLaube/AntClustering-IAR]`.

Como o projeto foi desenvolvido nativamente em ambiente Linux (Ubuntu), a execução exige a compilação via compilador GCC. Devido à dependência de operações matemáticas (como `sqrt` e `pow` presentes na biblioteca `<math.h>`), é estritamente necessário vincular a biblioteca matemática utilizando a *flag* `-lm`. Para reproduzir o experimento no terminal, utilize os seguintes comandos:

```
# Compilar o codigo-fonte da Fase 1
gcc antCol_txt_fase1.c -o antCol -lm

# Executar o simulador
./antCol
```

O primeiro experimento, materializado no arquivo da Fase 1, serviu como a fundação estrutural do simulador. O objetivo primário nesta etapa não era a clusterização de dados, mas sim a validação mecânica do ambiente artificial. O *grid* espacial foi implementado como uma matriz bidimensional simples operando em um plano toróide (onde as bordas se conectam, eliminando limites físicos). Nesta versão, adotou-se uma abordagem binária: as células da matriz armazenavam apenas o valor `0` (vazio) ou `1` (ocupado). A

percepção do ambiente pelas formigas foi programada de forma rudimentar, onde a função de "visão" consistia apenas em contar o número de itens na vizinhança imediata (um raio 3×3) e calcular a densidade bruta. O resultado empírico desse primeiro teste validou a mecânica de movimentação aleatória e as lógicas de pegar/largar, resultando na formação de aglomerados aleatórios de itens, comprovando que o comportamento emergente básico de empilhamento estava funcional.

Para monitorar essa execução massiva, foi desenvolvida uma rotina de instrumentação utilizando a biblioteca `<time.h>`. O algoritmo passou a reportar o progresso em saltos de 10%, calculando o tempo de uso de CPU gasto em cada etapa, culminando em um relatório descritivo no terminal. O resultado dessa arquitetura de software otimizada com a matemática de Lumer-Faieta pode ser visualizado na Figura 10, que demonstra o sucesso do experimento através da disposição final, onde os rótulos numéricos idênticos encontram-se clara e estavelmente agrupados no terminal de execução.

Figura 1. Disposição final dos clusters organizados e relatório de execução no terminal.

Contudo, para atingir o objetivo do projeto (Fase 2), uma reformulação profunda na estrutura de dados precisou ser feita no segundo experimento. A principal decisão de implementação recaiu sobre a matriz que representa o ambiente. Ao invés de uma matriz binária, o *grid* passou a ser inicializado com o valor `-1` (indicando espaço vazio) e, quando ocupado, passou a armazenar o **índice direto** do objeto correspondente contido no vetor de estruturas `Morta`. Essa escolha arquitetônica foi crucial para a performance do sistema: ela substituiu uma busca linear ineficiente ($O(N)$) por uma consulta instantânea ($O(1)$) na memória. Assim, quando uma formiga analisa um vizinho no grid, ela sabe exatamente qual objeto está lá e acessa instantaneamente suas coordenadas reais (`x_grupo` e `y_grupo`) lidas do arquivo `.txt`.

Com a infraestrutura de dados otimizada, a lógica matemática do algoritmo de Lumer-Faieta foi traduzida para a linguagem C. A função de visão foi inteiramente reescrita para calcular a similaridade baseada na Distância Euclidiana entre as propriedades

espaciais dos dados. Implementou-se a equação $f(i)$ que pondera a semelhança dos dados em relação ao fator de escala α. Posteriormente, a tomada de decisão das formigas foi substituída por uma "roleta probabilística": o sistema calcula as funções não-lineares de probabilidade de pegar (P_p) e soltar (P_d), baseadas nas constantes empíricas k_1 e k_2, e gera um número pseudoaleatório entre 0.0 e 1.0 (via `rand() / RAND_MAX`). Se o número sorteado for menor ou igual à probabilidade calculada, a formiga executa a ação.

3.1. Detalhamento das Fórmulas e Modelagem Probabilística

A função de similaridade $f(x_i)$ constitui o núcleo do algoritmo de Lumer-Faieta para dados heterogêneos. Diferentemente da abordagem da Fase 1, onde apenas a densidade era considerada, nesta etapa a similaridade é calculada com base na distância entre os atributos dos dados no espaço de características.

A distância entre dois dados x_i e x_j é definida pela distância euclidiana:

$$d(x_i, x_j) = \sqrt{(x_i^1 - x_j^1)^2 + (x_i^2 - x_j^2)^2} \tag{1}$$

onde x_i^1 e x_i^2 representam as dimensões do dado (neste trabalho, coordenadas bidimensionais).

A partir dessa definição, a função de similaridade local é dada por:

$$f(x_i) = \frac{1}{s^2} \sum_{x_j \in vizinhan\textbf{ç}a} \max\left(0, 1 - \frac{d(x_i, x_j)}{\alpha}\right) \tag{2}$$

onde:

- s^2 representa o número de células na vizinhança;
- α é um parâmetro de escala que controla a sensibilidade à distância;
- o operador $\max(0, \cdot)$ garante que apenas contribuições positivas sejam consideradas.

No código implementado, essa equação é refletida diretamente no cálculo da variável `f`, onde a similaridade entre pares de dados é acumulada e posteriormente normalizada pelo número de células vizinhas.

O parâmetro α exerce um papel fundamental: valores pequenos tornam o critério de similaridade mais restritivo, enquanto valores maiores tornam o algoritmo mais permissivo. Neste trabalho ele foi definido empiricamente com base na distribuição dos dados, e nos teste que foram feitos.

Uma vez obtida a similaridade $f(x_i)$, o comportamento das formigas é governado por duas funções probabilísticas:

$$P_p(x_i) = \left(\frac{k_1}{k_1 + f(x_i)}\right)^2 \tag{3}$$

$$P_d(x_i) = \left(\frac{f(x_i)}{k_2 + f(x_i)}\right)^2 \tag{4}$$

onde:

- $P_p(x_i)$ representa a probabilidade de uma formiga pegar um item;
- $P_d(x_i)$ representa a probabilidade de uma formiga soltar um item;
- k_1 e k_2 são constantes positivas que controlam a sensibilidade das funções.

Intuitivamente:

- Quando $f(x_i)$ é pequeno (item isolado), $P_p(x_i)$ tende a ser alto;
- Quando $f(x_i)$ é grande (item em grupo), $P_d(x_i)$ tende a ser alto.

A decisão final de executar uma ação é baseada em um processo estocástico: um valor aleatório $r \in [0, 1]$ é gerado e comparado com a probabilidade calculada. Caso $r \leq P$, a ação correspondente é executada.

3.2. Cenários de Experimento

Os experimentos foram divididos em duas categorias. Na primeira fase, validou-se o comportamento com dados homogêneos. Na segunda, realizou-se a calibração empírica dos hiperparâmetros para dados heterogêneos com 4 e 15 grupos. A Tabela 1 sumariza as configurações utilizadas.

Tabela 1. Parâmetros utilizados nos cenários de teste.

Parâmetro	Dados Homogêneos	4 Grupos	15 Grupos
Grid	50×50	64×64	64×64
Nº de Agentes	600	100	80
Nº de Dados	50	400	600
Raio de Visão (s)	1 (3×3)	1 (3×3)	1 (3×3)
Iterações	5.000.000	2.000.000	20.000.000
α	-	11.8029	0.95
k_1	-	0.3	0.05
k_2	-	0.6	1.1

3.3. Resultados Observados

Os resultados obtidos evidenciam o comportamento emergente esperado do algoritmo. Inicialmente, os dados encontram-se dispersos de forma aproximadamente uniforme. Com o avanço das iterações, pequenos agrupamentos locais começam a surgir, resultado da coleta de itens isolados pelas formigas.

Posteriormente, esses agrupamentos tornam-se progressivamente mais densos e organizados, culminando na formação de clusters bem definidos, nos quais dados similares encontram-se espacialmente próximos.

A utilização da distância euclidiana como métrica de similaridade mostrou-se eficaz na separação dos grupos, especialmente quando combinada com o fator de escala α. Entretanto, observou-se que a qualidade dos agrupamentos é sensível à escolha dos parâmetros α, k_1 e k_2, sendo necessária calibração empírica.

Além disso, o uso de um ambiente toroidal contribuiu para evitar efeitos de borda, proporcionando uma distribuição mais homogênea das interações entre os agentes.

4. Análise dos Resultados

Nessa seção será discutido os resultados obtidos durante os experimentos, todas as imagens dos resultados aqui apresentados também estão no disponíveis no repositório do github deste trabalho.

4.1. Resultado do experimento com dados homogêneos

Como pode ser observado na Figura 2, os dados são inicialmente distribuídos de forma aleatória no ambiente. Após a execução do algoritmo, verifica-se a formação de agrupamentos, conforme apresentado na Figura 3.

Esse resultado evidencia que o método está operando conforme o esperado, sendo capaz de organizar os dados a partir de interações locais simples. O tempo total de execução para esse experimento foi de aproximadamente 20,07 segundos.

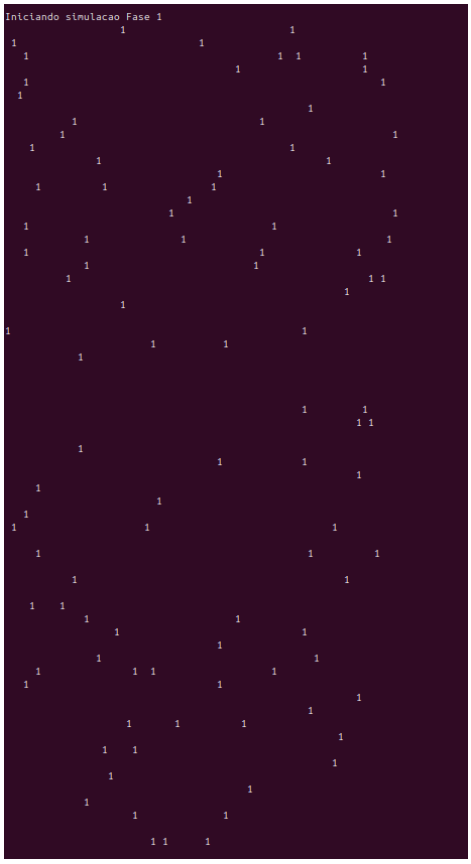

Figura 2. Disposição inicial (Homogêneos).

Figura 3. Disposição final (Homogêneos).

```
========================================
       RELATORIO FINAL - FASE 1
========================================
Iteracoes: 5000000 | Formigas Vivas: 100
Grid: 64 x 64 | Mortas: 101
Tempo Total: 20.0723 segundos
Desempenho: 0.00000401 s/iteracao
========================================
```

Figura 4. Parametros e métricas da simulação dados homogêneos.

4.2. Resultado do experimento com dados heterogêneos com 4 grupos

Na Figura 5, observa-se a distribuição inicial dos dados heterogêneos contendo quatro grupos distintos. Após a execução do algoritmo, verifica-se que os dados foram organizados em quatro clusters bem definidos, conforme mostrado na Figura 6.

Esse resultado demonstra a capacidade do algoritmo de separar corretamente grupos distintos com base na similaridade entre os dados. O tempo de execução observado para esse experimento foi de aproximadamente 17,10 segundos.

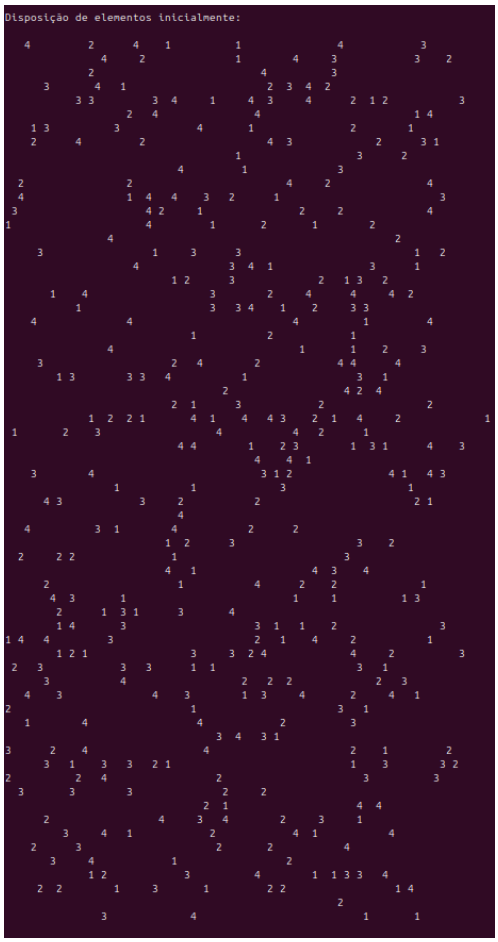

Figura 5. Disposição inicial (4 grupos).

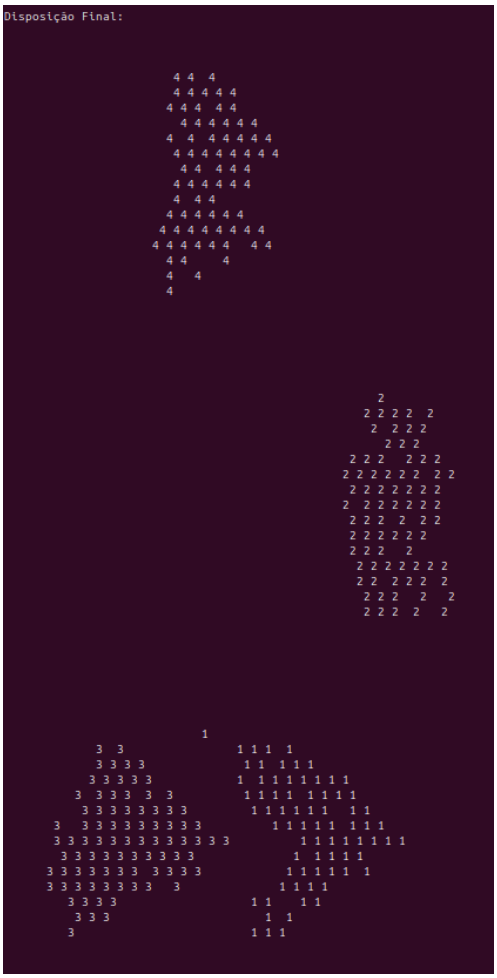

Figura 6. Disposição final (4 grupos).

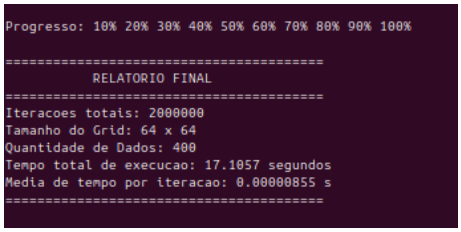

Figura 7. Parametros e métricas da simulação dados heterogêneos 4 grupos.

4.3. Resultado do experimento com dados heterogêneos com 15 grupos

No cenário com 15 grupos, o problema torna-se mais desafiador devido ao aumento da complexidade e da proximidade entre os diferentes clusters. Como pode ser observado na Figura 9, alguns agrupamentos encontram-se espacialmente próximos, podendo aparentar formar um único grupo maior. No entanto, uma análise mais detalhada revela que os clusters permanecem corretamente separados.

Em relação ao desempenho computacional, este foi o experimento mais demorado pois necessitou de mais interações para chegar em um resultado satisfatório, levando aproximadamente 305.63 segundos de processamento..

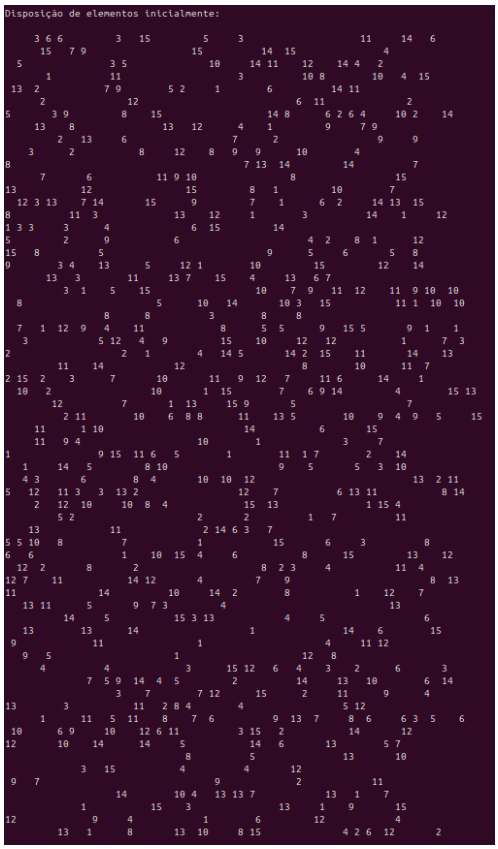

Figura 8. Disposição inicial (15 grupos).

Figura 9. Disposição final (15 grupos).

```
========================================
            RELATORIO FINAL
========================================
Iteracoes totais: 20000000
Tamanho do Grid: 64 x 64
Quantidade de Dados: 600
Tempo total de execucao: 305.6383 segundos
Media de tempo por iteracao: 0.00001528 s
========================================
```

Figura 10. Parametros e métricas da simulação dados heterogêneos 15 grupos.

5. Conclusões e Trabalhos Futuros

Este trabalho apresentou a implementação de um algoritmo de agrupamento baseado no comportamento de colônias de formigas, inspirado no modelo de Lumer-Faieta. O prin-

cipal objetivo foi investigar como regras locais simples, baseadas em percepção limitada e decisões probabilísticas, podem levar à formação de estruturas globais organizadas.

Na primeira etapa do projeto, com dados homogêneos, foi possível validar o funcionamento básico do sistema, evidenciando a emergência de agrupamentos a partir de uma abordagem puramente baseada em densidade. Essa fase foi fundamental para garantir a corretude da movimentação dos agentes e das regras de transição.

Na segunda etapa, com a introdução de dados heterogêneos, o algoritmo foi estendido para considerar similaridade baseada em distância euclidiana. Os resultados obtidos demonstraram que o modelo é capaz de separar dados em múltiplos grupos distintos, mesmo quando inicialmente distribuídos de forma aleatória. Em cenários com menor número de grupos, observou-se a formação de clusters bem definidos e espacialmente separados. Já em cenários mais complexos, com maior número de grupos, o algoritmo ainda foi capaz de organizar os dados corretamente, embora com maior proximidade entre alguns clusters.

Os experimentos também evidenciaram a forte dependência do desempenho do algoritmo em relação aos parâmetros utilizados, especialmente α, k_1 e k_2, bem como o raio de visão. Pequenas variações nesses parâmetros podem impactar significativamente a qualidade dos agrupamentos e o tempo de convergência, indicando a necessidade de calibração empírica.

Do ponto de vista computacional, observou-se que o custo de execução cresce consideravelmente com o aumento do número de dados, do número de iterações e do raio de visão, uma vez que este último influencia diretamente a quantidade de comparações realizadas na função de visão.

Como trabalhos futuros, destacam-se as seguintes possibilidades de evolução do projeto:

- Implementação de técnicas de normalização dos dados de entrada, visando melhorar a estabilidade da função de similaridade;
- Análise quantitativa dos agrupamentos, utilizando métricas como coeficiente de silhueta ou erro de clusterização;
- Métodos tradicionais como K-means e outras abordagens de clusterização são amplamente utilizados na literatura [Jain et al. 1999], servindo como base para comparação com métodos bio-inspirados;
- Paralelização do algoritmo, explorando o caráter independente dos agentes para redução do tempo de execução;
- Investigação de estratégias adaptativas para ajuste automático dos parâmetros α, k_1 e k_2.

Por fim, conclui-se que o algoritmo de agrupamento por colônia de formigas constitui uma abordagem robusta e biologicamente inspirada para problemas de clusterização, sendo capaz de gerar soluções coerentes a partir de interações locais simples, ainda que com custo computacional elevado e dependência de parâmetros.

Referências

Deneubourg, J.-L., Goss, S., Franks, N., Sendova-Franks, A., Detrain, C., and Chrétien, L. (1991). The dynamics of collective sorting: Robot-like ants and ant-like robots.

Proceedings of the First International Conference on Simulation of Adaptive Behavior, pages 356–365.

Jain, A. K., Murty, M. N., and Flynn, P. J. (1999). Data clustering: A review. *ACM Computing Surveys*, 31(3):264–323.

Lumer, E. D. and Faieta, B. (1994). Diversity and adaptation in populations of clustering ants. *Proceedings of the Third International Conference on Simulation of Adaptive Behavior*, pages 501–508.